Exercise 1: Implementing the Singleton Pattern

Scenario:

You need to ensure that a logging utility class in your application has only one instance throughout the application lifecycle to ensure consistent logging.

Code:

SingletonTest.java

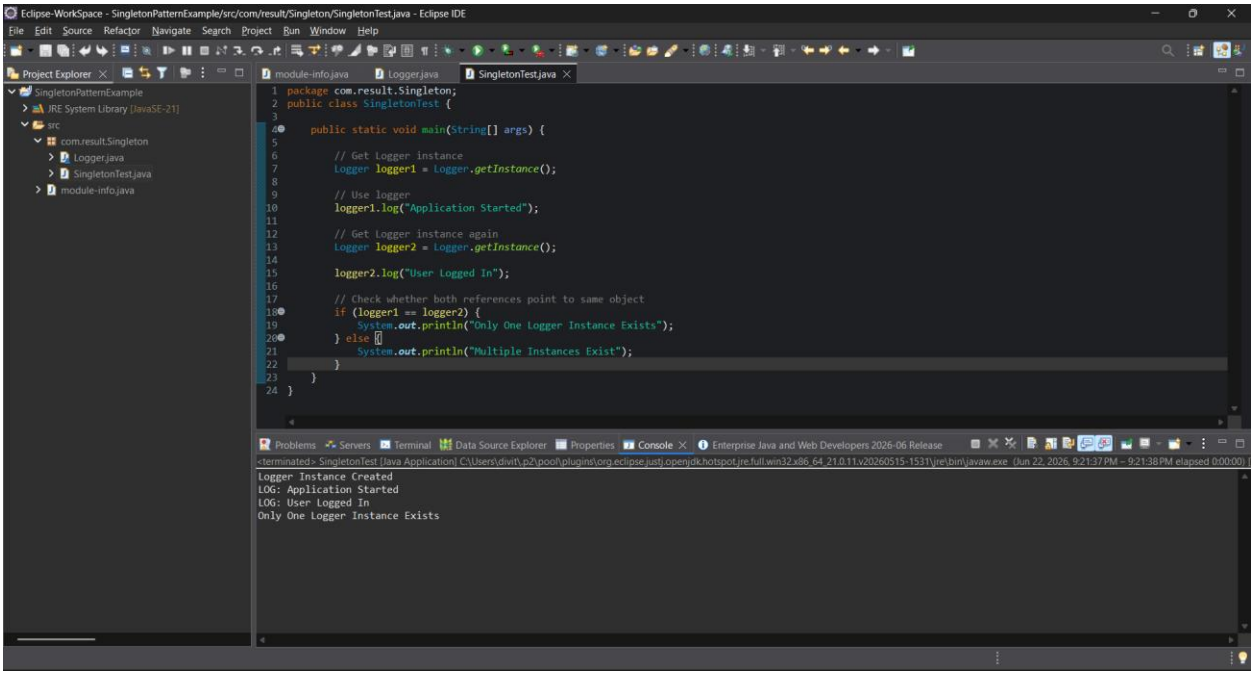
```
package com.result.Singleton;
public class SingletonTest {

    public static void main(String[] args) {

        // Get Logger instance
        Logger logger1 = Logger.getInstance();

        // Use logger
        logger1.log("Application Started");

        // Get Logger instance again
        Logger logger2 = Logger.getInstance();

        logger2.log("User Logged In");

        // Check whether both references point to same object
        if (logger1 == logger2) {
            System.out.println("Only One Logger Instance Exists");
        } else {
            System.out.println("Multiple Instances Exist");
        }
    }
}
```

Logger.java

```
package com.result.Singleton;
class Logger {

    // Step 1: Create a private static instance
    private static Logger instance;

    // Step 2: Private constructor
    private Logger() {
        System.out.println("Logger Instance Created");
    }

    // Step 3: Public method to get the instance
    public static Logger getInstance() {
        if (instance == null) {
            instance = new Logger();
        }
```

```
        return instance;
    }

    // Logging method
    public void log(String message) {
        System.out.println("LOG: " + message);
    }
}
```

Output:



Exercise 2: Implementing the Factory Method Pattern

Scenario:

You are developing a document management system that needs to create different types of documents (e.g., Word, PDF, Excel). Use the Factory Method Pattern to achieve this.

Document.java

```
package com.result.factory;

public interface Document {
    void open();
}
```

DocumentFactory.java

```
package com.result.factory;

public abstract class DocumentFactory {

    public abstract Document createDocument();}
```

ExcelDocument.java

```java
package com.result.factory;

public class ExcelDocument implements Document {

    @Override
    public void open() {
        System.out.println("Opening Excel Document");
    }
}
```

ExcelFactory.java

```java
package com.result.factory;

public class ExcelFactory extends DocumentFactory {

    @Override
    public Document createDocument() {
        return new ExcelDocument();
    }
}
```

PdfDocument.java

```java
package com.result.factory;

public class PdfDocument implements Document {

    @Override
    public void open() {
        System.out.println("Opening PDF Document");
    }
}
```

PdfFactory.java

```java
package com.result.factory;

public class PdfFactory extends DocumentFactory {

    @Override
    public Document createDocument() {
        return new PdfDocument();
    }
}
```

WordDocument.java

```java
package com.result.factory;

public class WordDocument implements Document {

    @Override
    public void open() {
        System.out.println("Opening Word Document");
    }
}
```

WordFactory.java

```java
package com.result.factory;

public class WordFactory extends DocumentFactory {

    @Override
    public Document createDocument() {
        return new WordDocument();
    }
}
```

Output: